ECE391
Computer System Engineering

Lecture 22

University of Illinois at Urbana- Champaign

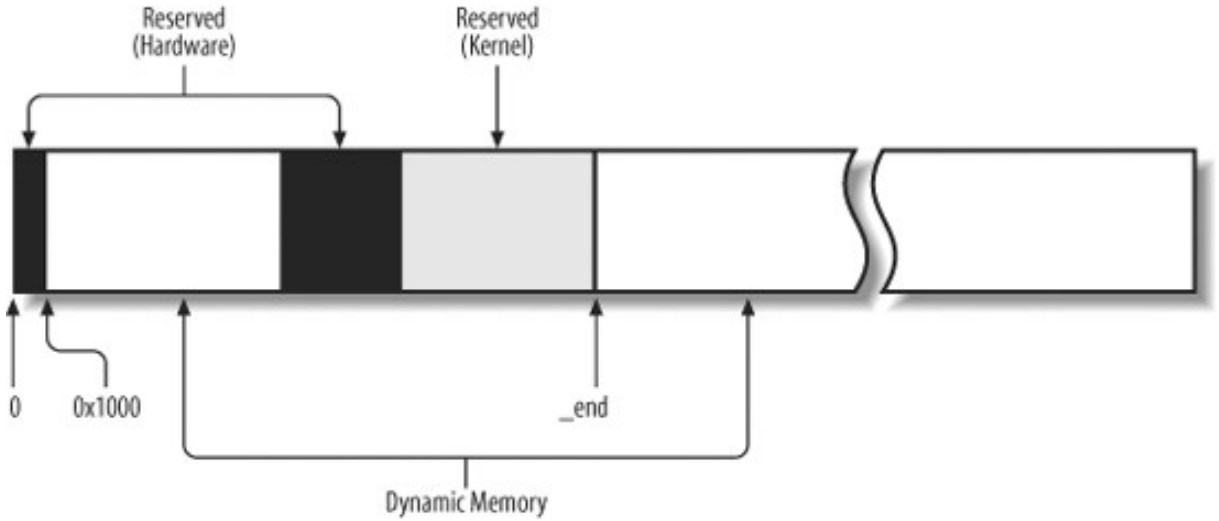
Spring 2021

Memory Management

- Need to manage two things:
  - Virtual address space
  - Physical memory
- Several flavors of physical memory on x86:
  - Reserved by hardware
  - Available memory
  - DMA-accessible memory
    - Subset of regular memory
- Part of memory occupied by kernel, rest is available for dynamic allocation

Linux Dynamically-Allocated Physical Memory Flavors (Types)

- **Direct mapped memory:** physical memory directly mapped into the kernel address space
  - Older devices can only directly access lowest 16MB, so allocations for DMA come from here
- **High memory:** memory above 896MB that is not accessible via direct mapping



Direct Memory Access

- Data from slow devices (e.g. UART) accessed by reading I/O registers when data becomes availabe
- Some devices (hard drives, network cards) transfer large amounts of memory
  - Would be time-consuming to read it one byte at a time from some I/O register as data becomes available
- Direct Memory Access (DMA) allows devices to write to physical memory directly
- Device driver allocates memory for DMA transfer
  - Buffer must by physically contiguous
- Configures device to write to the physical address of buffer
- Devices operate on physical addresses (why?)

Dynamic Allocation Physical Memory Zones

- Kernel manages memory in three separate *zones:*
  - **DMA zone:** for DMA allocations
    excluding hardware reserved and kernel image
  - **Normal zone:** lowest 896 MB mapped into kernel VA space
  - **High memory zone:** memory above 896 MB
    - Not directly-mapped in kernel virtual memory space
- Is this just Linux and x86 weirdness?
  - Yes and No: other architectures may have similar constraints

Page Descriptors

- Each page frame (physical memory page) has an associated 32-byte page descriptor (struct page)
  - mem_map points to array of struct page
  - max_mapnr is number of page frames in array
  - Page frame number *n* is at address *n* × 4096, descriptor is `mem_map[`*n*`]`

Allocating Page Frames

- `alloc_pages(gfp_mask, order)`: allocate a contiguous range of 2^{order} page frames
- `alloc_page(gfp_mask)`: defined as `alloc_pages(gfp_mask, 0)`
- `__get_free_pages(gfp_mask, order)`: like `alloc_pages`, but returns linear address of first page
  - Cannot be used for high-memory pages (why?)
- `__free_pages(page, order)`: release page frames
  - Note: order must be same as used to allocate!

Memory Allocation Flags

may sleep to wait for pages

GFP_KERNEL	by kernel, drivers, etc.
GFP_NOFS	no file system calls (avoids pushing pages to disk)
GFP_NOIO	no I/O operations at all
GFP_USER	on behalf of a user (low priority)

__GFP_DMA	DMA accessible (low physical addresses on some machines)
__GFP_HIGHMEM	high memory (PAE (phys. address extensions) on x86) is acceptable

(two underscores)

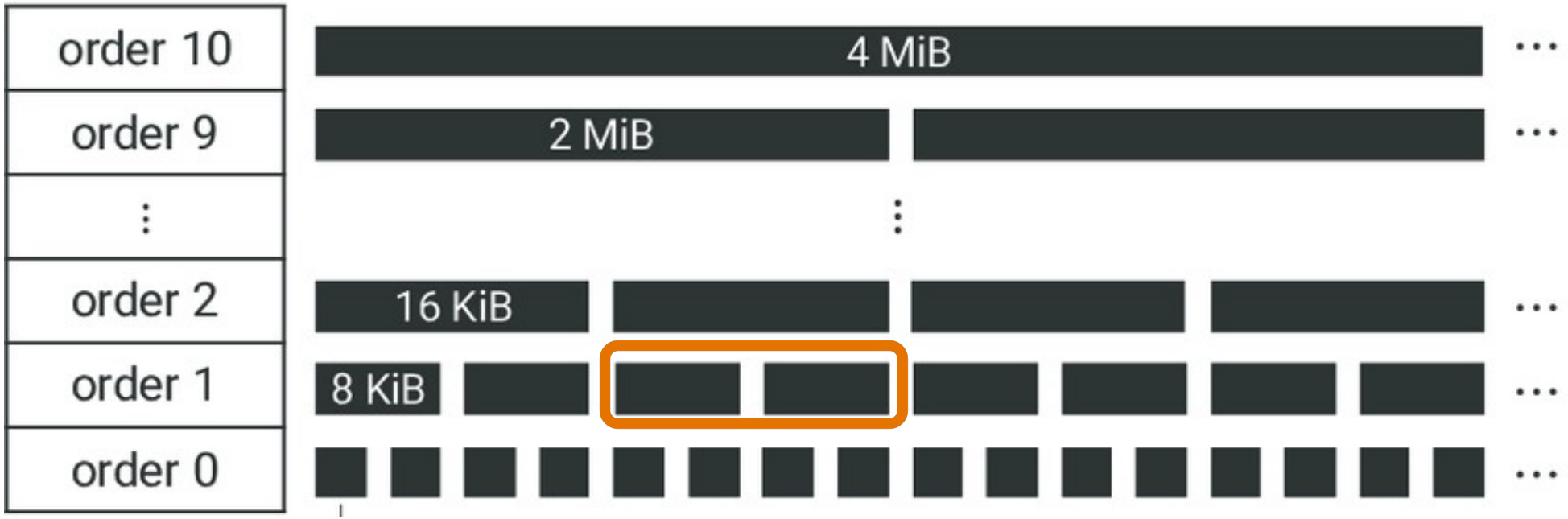
Zone Allocation

- Flags passed to `alloc_pages()` and related functions specify which zone to allocate from:
  - If `__GFP_HIGHMEM` is set, try to allocate from high memory zone first, then normal zone, then from DMA zone
  - Otherwise, if `__GFP_DMA` not set, try to allocate from normal zone first, then from DMA zone
  - Otherwise, try to allocate from DMA zone only

Buddy System

- Manage memory as blocks of memory of sizes that are a power-of-two multiple of page size
  - E.g. 4K blocks, 8K, blocks, 16K, blocks, 32K, blocks, etc.
- Each block has a buddy, which is the adjacent block with which it makes up the next larger size bloc

Buddy System Blocks



K. Levchenko

Buddy System Allocation

- All allocation requests are for power-of-two multiples of the smallest block size (page for zone allocator)
- Maintain list of free blocks of each power-of-two size
- To allocate a block, check free list
- If no free block of size *B* available, get a block of size 2*B* and split it in half
  - If no block of size 2*B*, try 4*B*, etc.

Buddy System Freeing Block

- When a block is freed, check if its buddy is also free
- For each pair of buddies, need 1 bit of information:
  - 0: if both buddies free or allocated
  - 1: if exactly one buddy is free
- If both buddies now free, merge into larger block
  - Continue merging larger blocks if necessary

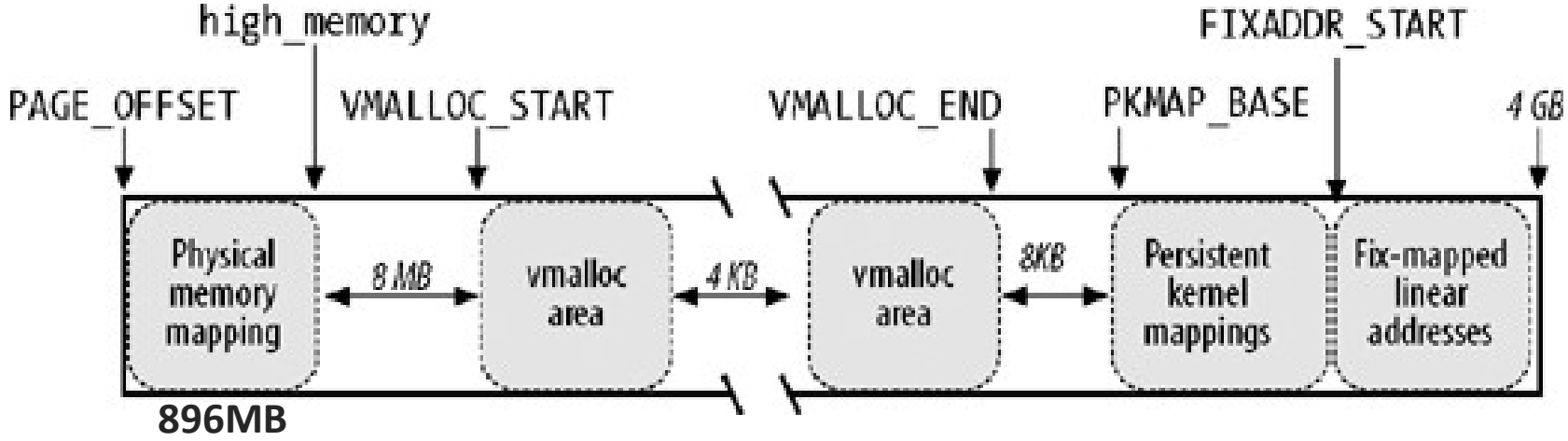
Buddy System

- Two data structures:
  - Buddy status bitmap (how many bits total?)
  - Free block list (store links in block itself)
- Allocation time: O(*L*) where *L* is number of sizes
  - *L* is at most $\log_2$[(size of memory) / (page size)]
- Deallocation time: O(*L*) also

Allocating Page Frames

- `alloc_pages(gfp_mask, order)`: allocate a contiguous range of 2^{order} page frames
- `alloc_page(gfp_mask)`: defined as `alloc_pages(gfp_mask, 0)`
- `__get_free_pages(gfp_mask, order)`: like `alloc_pages`, but returns linear address of first page
  - Cannot be used for high-memory pages (why?)
- `__free_pages(page, order)`: release page frames
  - Note: order must be same as used to allocate!

Linux Kernel Virtual Address Space



- **PAGE_OFFSET:** Virtual address that maps to first page frame of physical memory (usually `0xC0000000`)
- **VMALLOC_START to VMALLOC_END:** Address range used by `vmalloc()` allocator of physically non-contiguous memory
- **PKMAP_BASE:** Dynamic mapping of high physical memory

kmalloc()

```
void* kmalloc (size_t size, gfp_t flags);
```

- Allocates memory from low memory zone
  - Can ask for DMA-accessible memory in lowest 16MB
- Returns a virtual address accessible in kernel
  - Recall that low memory is 1:1 mapped in kernel, so no need to modify page tables
- Under the hood uses caches of pre-allocated power-of-two-sized blocks to satisfy requests

vmalloc()

```
void* vmalloc (size_t size);
```

- May allocate page frames from high memory
- Returns a virtual address from VMALLOC region
- *Memory may not be physically contiguous*
- Updates page table: requires TLB flush

Memory Fragmentation

- ***External fragmentation:*** Frequent allocation/deallocation of group of continuous pages in physical memory of different sizes may lead to situation in which several small blocks of free pages are scattered inside blocks of allocated pages. As a result it may be impossible to allocate a large block of contiguous pages even if there is enough free pages.

- ***Internal fragmentation:*** caused by a mismatch between the size of the memory request and the size of the memory area allocated to satisfy the request

User-Space Malloc

- The C library provides malloc(size_t size) for memory allocation by user programs
- Memory allocated from heap
- Heap can grow using brk() or sbrk() system calls
  - Kernel modifies process memory map to increase heap
  - May defer allocation of page frames to back new pages
- Modern implementations use mmap() system call to map blocks of memory in process address space
- In either case, malloc() gets large chunks of memory from the kernel and allocates memory from those

Malloc Implementation

- Track regions of free memory
  - Linked list, use memory block itself to store links
- Allocate memory from first region that is large enough
  - Remaining space in region becomes new free memory block
  - Merge regions if two adjacent regions freed
- Finding free region of correct size may be slow

- Buddy system
  - But free(void * p) does not supply initial allocation size, so need to keep track of that information too